```
# The race
while True:
    n = random.randint(0,9)
    turtle = turtles[n]
    turtle.forward(1)
    if turtle.xcor() == 100:
        print("Turtle %d wins"%(n+1))
        break
```

Scratch

Scratch is the common option for Linux. Scratch 2 is not easy to run on Linux as it uses AdobeAIR, which is no longer available for Linux. The major difference between the two is that Scratch 2 allows the creation of custom blocks, which will be discussed later.

Scratch uses blocks grouped into various categories for constructing programs. A block may have parameters. It is easy to drag and connect various blocks to construct a program visually.

The key components of Scratch are sprites, costumes, sounds and scripts. A sprite is a graphic object (perhaps a virtual elf or fairy?), which is manipulated as an entity. Each sprite can be represented by different shapes, called costumes. Sounds are exactly what you may expect.

One or more scripts can be applied to each sprite. The triggering of each script is via an event. Hence, Scratch is a great way to get exposed to event-driven programming.

The example shown in Figures 2, 3 and 4 uses the default sprite with two costumes and a sound. Add three backgrounds to the stage. In the example, the sprite alternates between costumes and moves forward, giving the illusion of walking. Once it hits the end of the stage, it goes to the other end and sends a message to change the background. The sound is triggered by pressing the space bar.

Figure 2 is the script for the sprite and Figure 3 is the script for the stage. Figure 4 shows a few scenes of the stage.

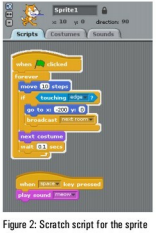

Figure 2: Scratch script for the sprite

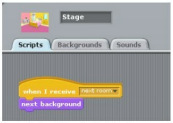

Figure 3: Scratch script for the stage

Snap!

This was a project at the University of California Berkeley —to add custom blocks to MIT's Scratch. It was initially an enhancement of Scratch and called BYOB (build your own block). Meanwhile, Scratch 2 was released, offering the same functionality, though written in AdobeAIR. Snap! is a new implementation modelled on Scratch in JavaScript.

Figure 4: Scratch stage scenes

It is expected to be used online; however, the source can be downloaded and installed on a local machine as follows:

```
$ wget http://snap.berkeley.edu/snapsource/snap.zip
$ cd ~/public_html
$ unzip ~/snap.zip
```

Apache should be running and user directories should be enabled. For example, on Fedora, it is in the file */etc/httpd/conf.d/userdir.conf*.

Browse *http://localhost/~anil/*.

The source does not have a library of sample sounds, costumes and backgrounds. However, that is not a limitation. You can drag and drop these from the Scratch installation in */usr/share/scratch/Media*.

In a revised implementation of the example above, create a block *Walk* with the number of steps as a parameter. The block issues the message 'Next room' once it reaches the end.

Figure 5 shows the block definition and the sprite scripts. There is no change in the stage script from the Scratch example.

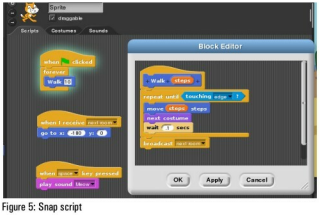

Figure 5: Snap script

You can clone sprites and write recursive code in custom blocks. Each of the environments allows a student to learn and create complex projects, which is fun.

The wonderful aspect of programming is that you learn by doing. And open source software is truly amazing! END

By: Dr Anil Seth

The author has earned the right to do what interests him. You can find him online at *http://sethanil.com*, *http://sethanil.blogspot.com*, and reach him via email at *anil@sethanil.com*